

2025-09-03: Due to updates GitLab may be unavailable for some minutes between 13:00 and 16:00.

Project information

Created on

October 28, 2024



Add `--cleanenv` flag for apptainer

[Tobias Ribizel](#) authored 1 month ago

Name	Last commit	Last update
.gitlab/issue_templates	indicate people should not use LRZ account...	1 month ago
README.md	Add <code>--cleanenv</code> flag for apptainer	1 month ago

[README.md](#)

Sections:

- [Connecting to the cluster](#)
- [Login node](#)
- [Accessing compute nodes](#)
- [Available hardware](#)
- [Software](#)
- [Containers](#)
- [Data storage](#)
- [Access for external users](#)
- [Service requests](#)

Connecting to the cluster

The cluster is available only inside the Münchner Wissenschaftsnetz (MWN), you need to use eduVPN if you are working from home or through eduroam. You can connect to the server via ssh:

```
ssh username@cluster.ginkgo-project.de
```

There are some DNS issues when using eduVPN and eduroam that we are currently trying to resolve with campus IT, so if you can't connect to the server by its hostname, try

```
ssh username@10.152.225.230
```

If you need to change your ssh keys, open an issue [here](#)

Login node

When you connect to the cluster, you are on the login node. This node is used by all users at the same time, so please be mindful of how you use it:

- Use the login node for short pre- or post-processing, compiling small projects, developing code, downloading data, ...
- Please don't execute any long-running and CPU-heavy processes, use the compute nodes for that purpose.
- All of your processes will be killed once you log out of your last session, `tmux` or `nohup` will not prevent this.

Accessing compute nodes

We use [Slurm](#) to manage access to the compute nodes. Accessing a compute node happens through **jobs** that are submitted to a *queue*.

```
# anatomy of a salloc call:
salloc [-w hostname] [--gres gpu:vendor:count] [--exclusive] [-n number-of-cores] [-t duration]

# run an interactive job on any available node
# This requests only 1 CPU core and no GPUs for 1 hour (default)
# The CPU limits are not enforced, but the GPUs are not available
salloc

# run an interactive job on a specific node with a 2h time limit
# and a single AMD GPU
salloc -w gpu-amd -t 02:00:00 --gres gpu:amd:1

# allocate an entire node with 2 GPUs for yourself for 30 minutes
# please only do this if you need to run benchmarks!
salloc --exclusive --gres gpu:nvidia:2 -t 00:30:00

# allocate 24 tasks (= cores) on any node
# this limit is not enforced, but prevents others from allocating these cores
salloc -n 24
```

If you don't want to run an interactive job, you can use `sbatch` to submit a **batch job**, which takes an input script and submits it to a queue.

You can also find some more usage information about Slurm, especially how to write batch job files, at the [LRZ documentation](#).

Available hardware

Hostname	CPU	Memory	Storage	GPUs (and how to allocate them)	Purpose
login	1x AMD EPYC 9124 (16C/32T)	128 GB	880 GB NVMe SSD	--	login
gpu-nvidia	2x Intel Xeon Gold 6438Y+ Sapphire Rapids (32C/64T)	1 TB	880 GB NVMe SSD	2x NVIDIA L40S (48 GB) --gres gpu:nvidia:1	compute
gpu-amd	2x Intel Xeon Gold 6438Y+ Sapphire Rapids (32C/64T)	256 GB	880 GB NVMe SSD	2x AMD MI210 (64 GB) --gres gpu:amd:1	compute
gpu-intel-pvc	2x Intel Xeon Platinum 8480+ Sapphire Rapids (56C/112T)	512 GB	7 TB NVMe SSD	2x Intel Max 1100 (48 GB) --gres gpu:intel:1	compute
gpu-nvidia-h100	2x Intel Xeon Gold 6548N Sapphire Rapids (32C/64T)	2 TB	15 TB NVMe SSD	4x NVIDIA H100 (96 GB) --gres gpu:nvidia:1	compute
gpu-intel	1x AMD Ryzen 9 7900X3D (12C/24T)	128 GB	916 GB NVMe SSD	2x Intel A770 (16 GB) --gres gpu:intel:1	development
rocinante	2x AMD EPYC 7713 Milan (64C/128T)	512 GB	880 GB NVMe SSD	1x NVIDIA A100 (40 GB) 2x NVIDIA A2 (16 GB) 1x NVIDIA P100 (16 GB) 3x AMD MI50 (16 GB) 1x Intel A770 (16 GB) --gres gpu:nvidia/amd/intel:1	development

Software

The software on the compute nodes is managed via [Spack](#), which is a package manager focused on High Performance Computing environments.

We expose the spack packages via modules, which can be listed through the `module avail` command and loaded with the `module load` command.

```
# list all available modules
$ module avail
-----
rocm/5.1.2    rocm/5.2.4    rocm/5.3.0    rocm/5.3.3    rocm/5.4.1    rocm/5.4.3    rocm/5.4.6    rocm/5.5.1
LRZ Homepage | Datenschutz | Dokumentation und Betriebsbedingungen | Impressum
```

```
----- /storage/app:
  aptainer/1.3.4      cmake/3.25.3      cuda/11.1.1      cuda/12.1.1      gcc/9.5.0      git/2.45.2
  cmake/3.16.9        cmake/3.26.6      cuda/11.2.2      cuda/12.2.2      gcc/10.5.0     googletest/1.14
  ...

# list all cmake versions
$ module avail cmake
----- /storage/app:
  cmake/3.16.9  cmake/3.18.6  cmake/3.19.8  cmake/3.20.6  cmake/3.21.7  cmake/3.22.6  cmake/3.23.5

# load the default version of cmake
$ module load cmake
# load a specific version of cmake
+-----
```

If you think there is any software missing that you and others might need, please open an issue [here](#)

If you are using Python-based software, you can create your own environment with [venv](#) and `pip`, or [conda](#) to install your own software without administrator privileges.



repositories only. Anaconda is trying to collect license fees from academic institutions where people use it. [Details](#)

Containers

Complex software environments can often most easily be stored via containers. We support the Apptainer/Singularity container runtime, which runs entirely in user-space, and has built-in support for GPUs. The commands you need to know:

Downloading a container

To pull a container from DockerHub and write it to a Singularity Image File (SIF)

```
apptainer pull docker://ubuntu:latest # creates ubuntu_latest.sif
```

Open a shell inside the container

Once you dowloaded the SIF file, you can enter it via

```
apptainer shell ubuntu_latest.sif
```

Inside an apptainer shell, your home directory is mounted regularly, but all software etc. is provided by the container. Note that all files outside the home directory are read-only.

GPU support

If you want to use an NVIDIA GPU with the application inside the container, pass the `--nv` flag

```
apptainer shell --nv tensorflow_latest.sif
```

Mounting additional paths

If you want to access additional files outside the home directory from the host, you can specify their path using the `--bind` flag. The parameter can take three forms: `bind-path`, `bind-source:bind-destination` or `bind-source:bind-destination:rw/ro`

```
# mount /tmp as /tmp, mount /data as /container-data with read-only access
apptainer shell --bind /tmp --bind /data:/container-data:ro alpine_latest.sif
```

Writable containers and Fakeroot

If you need to modify some files in the container, you can add a persistent overlay, either as a separate file, or inside the container. For more details, check out https://apptainer.org/docs/user/latest/persistent_overlays.html

If you want to become root inside the container (which doesn't give you actual root permissions, just lets you write to all files inside the container), you can use the `fakeroot` feature of apptainer. For more details, check out <https://apptainer.org/docs/user/latest/fakeroot.html#usage>

```
# create a 1GB overlay for all changed/added files on top of the container
apptainer overlay create --fakeroot --size 1024 overlay.img
# enter the container with the overlay mounted
apptainer shell --cleanenv --fakeroot --overlay overlay.img ubuntu_latest.sif
# now you can install your own software
apt-get update
apt-get install git
```

Data storage

Each user has their own `home` directory that is part of our storage node, mounted as a network filesystem (NFS). The home directories get snapshotted and backed up to LRZ daily. If you accidentally deleted a file, we will most likely be able to recover it, if it was created more than a day ago.

We have 60 TB of storage available for software, user homes and research data, but we should try to keep it below 20 TB for backup purposes.

Each node also has a small amount of fast local `scratch` storage that is symlinked to your home directory's scratch path. This storage is not available from other nodes and will not be backed up. If we run into storage limitations, we will clean up this scratch storage, so don't use it for anything permanent.

Access for external users

External users can log in through our jump host without the need for a VPN:

```
ssh -4 -p 55841 username@comavm1.cit.tum.de
```

This access path is only enabled for external users. Alternatively, they can use an ssh config entry like

```
Host coma-cluster
  Hostname comavm1.cit.tum.de
  User username
  Port 55841
  AddressFamily inet
  IdentityFile ~/.ssh/id_rsa
  IdentitiesOnly yes
```

and log in via

```
ssh coma-cluster
```

Service requests

1. [Create a new user account for a TUM employee](#)
2. [Create a new user account for a TUM student](#)
3. [Create a new user account for an external collaborator](#)
4. [Change your ssh authorized_keys](#)
5. [Deactivate a user account](#)
6. [Software request](#)
7. [Other issue or request](#)